QUESTION 5

```
import random

# --------------------------------
# (a) Representation + Fitness
# --------------------------------

def random_chromosome():
    return [(random.randint(0,2), random.randint(0,3)) for _ in range(6)]

def count_conflicts(chromosome):
    conflicts = 0
    n = len(chromosome)

    for i in range(n):
        for j in range(i+1, n):
            if chromosome[i] == chromosome[j]:
                conflicts += 1

    return conflicts

def fitness(chromosome):
    return 100 - (10 * count_conflicts(chromosome))


def print_random_samples():
    print("\n--- Random Chromosomes ---")
    for i in range(5):
        c = random_chromosome()
        conf = count_conflicts(c)
        fit = fitness(c)
        print(f"{c} | Conflicts={conf} | Fitness={fit}")


# --------------------------------
# (b) Crossover, Repair, Mutation
# --------------------------------

def crossover(p1, p2, point):
    c1 = p1[:point] + p2[point:]
    c2 = p2[:point] + p1[point:]
    return c1, c2
```

```
def repair(chromosome):
    seen = set()
    new_chromosome = []

    for gene in chromosome:
        if gene not in seen:
            new_chromosome.append(gene)
            seen.add(gene)
        else:
            # find conflict-free assignment
            possible = [(r, s) for r in range(3) for s in range(4)]
            random.shuffle(possible)

            assigned = False
            for p in possible:
                if p not in seen:
                    new_chromosome.append(p)
                    seen.add(p)
                    assigned = True
                    break

            if not assigned:
                # fallback (no free slot)
                new_gene = random.choice(possible)
                new_chromosome.append(new_gene)
                seen.add(new_gene)

    return new_chromosome


def mutate(chromosome, pm):
    new_chromosome = []
    for gene in chromosome:
        if random.random() < pm:
            new_gene = (random.randint(0,2), random.randint(0,3))
            new_chromosome.append(new_gene)
        else:
            new_chromosome.append(gene)
    return new_chromosome


# Demonstration for part (b)
def demo_crossover_and_repair():
    print("\n--- Crossover Conflict Demo ---")
```

```
    p1 = [(0,0),(0,0),(1,1),(2,2),(1,2),(2,3)]
    p2 = [(0,0),(1,1),(2,2),(0,0),(1,3),(2,0)]

    print("Parent 1:", p1)
    print("Parent 2:", p2)

    c1, c2 = crossover(p1, p2, 3)

    print("\nAfter Crossover:")
    print("Child 1:", c1, "Conflicts:", count_conflicts(c1))
    print("Child 2:", c2, "Conflicts:", count_conflicts(c2))

    print("\nAfter Repair:")
    r1 = repair(c1)
    r2 = repair(c2)

    print("Repaired Child 1:", r1, "Conflicts:", count_conflicts(r1))
    print("Repaired Child 2:", r2, "Conflicts:", count_conflicts(r2))


def demo_manual_repair():
    print("\n--- Manual Repair Demo ---")

    c = [(0,0),(0,0),(1,1),(1,1),(2,2),(2,2)]

    print("Before:", c, "Conflicts:", count_conflicts(c))
    repaired = repair(c)
    print("After :", repaired, "Conflicts:", count_conflicts(repaired))


# --------------------------------
# (c) Full GA (Tournament Selection)
# --------------------------------

def tournament_select(population):
    a = random.choice(population)
    b = random.choice(population)
    return a if fitness(a) > fitness(b) else b


def run_scheduling_ga(pop_size, generations, pm):
    population = [random_chromosome() for _ in range(pop_size)]
    best_history = []
```

```
    for gen in range(generations):

        new_population = []

        for _ in range(pop_size // 2):
            p1 = tournament_select(population)
            p2 = tournament_select(population)

            point = random.randint(1, 5)

            c1, c2 = crossover(p1, p2, point)

            # Repair after crossover
            c1 = repair(c1)
            c2 = repair(c2)

            # Mutation
            c1 = mutate(c1, pm)
            c2 = mutate(c2, pm)

            new_population.extend([c1, c2])

        population = new_population

        # Track best
        best = max(population, key=fitness)
        best_fit = fitness(best)
        best_history.append(best_fit)

        if count_conflicts(best) == 0:
            print(f"\nSolution found at generation {gen}: {best}")
            break

    # Final best
    best = max(population, key=fitness)

    print("\n--- Final Best Schedule ---")
    print(best)
    print("Conflicts:", count_conflicts(best))
    print("Fitness:", fitness(best))


# ------------------------------
```

```
# Main Execution
# ------------------------------

if __name__ == "__main__":

    # (a)
    print_random_samples()

    # (b)
    demo_crossover_and_repair()
    demo_manual_repair()

    # (c)
    print("\n--- Running Scheduling GA ---")
    run_scheduling_ga(pop_size=20, generations=50, pm=0.1)
```

```
 --- Random Chromosomes ---
 [(1, 1), (0, 0), (2, 3), (1, 0), (0, 1), (1, 2)] | Conflicts=0 | Fitness=100
 [(1, 3), (2, 3), (2, 0), (2, 2), (2, 0), (0, 0)] | Conflicts=1 | Fitness=90
 [(0, 1), (2, 1), (1, 3), (2, 1), (2, 1), (1, 1)] | Conflicts=3 | Fitness=70
 [(2, 1), (2, 1), (1, 3), (0, 2), (0, 1), (1, 3)] | Conflicts=2 | Fitness=80
 [(1, 1), (2, 0), (2, 0), (1, 3), (1, 0), (0, 0)] | Conflicts=1 | Fitness=90
```

```
--- Crossover Conflict Demo ---
Parent 1: [(0, 0), (0, 0), (1, 1), (2, 2), (1, 2), (2, 3)]
Parent 2: [(0, 0), (1, 1), (2, 2), (0, 0), (1, 3), (2, 0)]

After Crossover:
Child 1: [(0, 0), (0, 0), (1, 1), (0, 0), (1, 3), (2, 0)] Conflicts: 3
Child 2: [(0, 0), (1, 1), (2, 2), (2, 2), (1, 2), (2, 3)] Conflicts: 1

After Repair:
Repaired Child 1: [(0, 0), (1, 0), (1, 1), (2, 0), (1, 3), (2, 3)] Conflicts: 0
Repaired Child 2: [(0, 0), (1, 1), (2, 2), (0, 3), (1, 2), (2, 3)] Conflicts: 0

--- Manual Repair Demo ---
Before: [(0, 0), (0, 0), (1, 1), (1, 1), (2, 2), (2, 2)] Conflicts: 3
After : [(0, 0), (2, 0), (1, 1), (0, 2), (2, 2), (1, 3)] Conflicts: 0

--- Running Scheduling GA ---

Solution found at generation 0: [(0, 2), (1, 3), (0, 1), (0, 0), (2, 2), (2, 3)]

--- Final Best Schedule ---
[(0, 2), (1, 3), (0, 1), (0, 0), (2, 2), (2, 3)]
Conflicts: 0
Fitness: 100
```

(d) Analysis

The Genetic Algorithm was able to find a conflict-free schedule quickly, sometimes even in the initial population. This indicates that valid solutions are not extremely rare in the search space. However, the problem becomes harder as the number of courses increases because the number of possible assignments grows exponentially while the proportion of valid schedules decreases.

The main constraint — no two courses sharing the same room and time slot — creates conflicts that reduce the number of feasible solutions. As more courses are added, the likelihood of collisions increases, making it harder for random assignments to be valid.

Genetic Algorithms are preferable to Random Restart Hill Climbing (RRHC) when:

1. The search space is large and diverse, as GA maintains a population and explores multiple regions simultaneously.
2. The problem has many local optima, since crossover allows combining good partial solutions.

RRHC is preferable when:

1. The constraint density is low and good solutions are easy to reach from random starts.
2. The search space is small or smooth, where simple hill climbing converges faster without the overhead of maintaining a population.